

Log-Normal Multiplicative Dynamics for Stable Low-Precision Deep Learning

Keigo Nishida^{*12} Eren Mehmet Kiral²³ Kenichi Bannai²³ Mohammad Emtiyaz Khan² Thomas Möllenhoff²

Abstract

We propose Log-Normal Multiplicative Dynamics (LMD), a new algorithm to stabilize low-precision deep learning. LMD is motivated by the robustness of biological neural networks where synaptic spine sizes follow a log-normal distribution and fluctuate via noisy multiplicative dynamics. So far, no algorithm has successfully incorporated all these synaptic properties to train large models. LMD incorporates these properties through a variational Bayesian formulation that uses a log-normal posterior over weights. We evaluate LMD with low-precision forward-pass matrix multiplications, including training a Vision Transformer and GPT-2 from scratch. Our findings demonstrate the utility of biologically inspired approaches for designing energy-efficient algorithms for stable low-precision deep learning.

1. Introduction

Low-precision deep learning is key to efficient deep learning in both training and inference under constrained computational and power budgets. In particular, converting weights and activations to low-precision data formats can accelerate matrix multiplications using specialized low-precision hardware engines (Micikevicius et al., 2018; Wang et al., 2018; NVIDIA et al., 2025; Wang et al., 2025). Such formats reduce bit widths at the expense of increased numerical error, while preserving as much dynamic range as possible (Kalamkar et al., 2019; Micikevicius et al., 2022; Rouhani et al., 2023a). Because the numerical errors introduced by such formats typically scale with magnitude, low-precision deep learning requires algorithms that are robust to magnitude-dependent fluctuations. From the perspective of the update rule, multiplicative weight updates (MWUs) are naturally aligned with this requirement because

^{*}Corresponding author ¹RIKEN Center for Biosystems Dynamics Research, Kobe, Japan ²RIKEN Center for Advanced Intelligence Project, Tokyo, Japan ³Keio University, Yokohama, Japan. Correspondence to: Keigo Nishida <keigo.nishida.jg@riken.jp>.

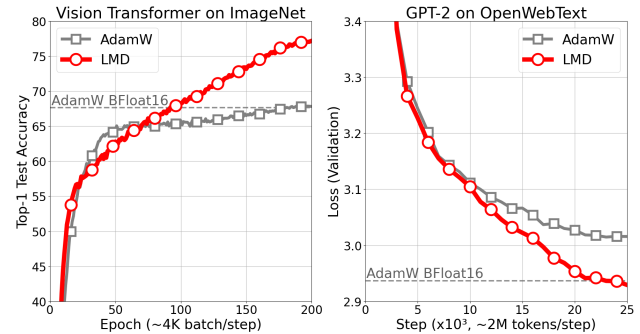


Figure 1. LMD trains Vision Transformer (ViT) and GPT-2 from scratch under low-precision (MXFP6) forward passes. AdamW performs worse than LMD. For Madam, a previously proposed scalable multiplicative update, training fails (see Tables 1 and 2). For GPT-2, all methods use the same number of tokens per step. LMD and Madam use a sequence length of 4096 and a batch size of 16; AdamW uses 1024 and 64.

their updates scale with the magnitude of each weight (Bernstein et al., 2020; Zhao et al., 2022). However, existing MWU methods face scalability challenges and have not yet demonstrated successful training of modern deep networks from scratch, such as Vision Transformers (ViTs) (Dosovitskiy et al., 2021) or GPT-2 (Radford et al., 2019). In addition, existing MWU methods do not explicitly address low-precision activations. These limitations motivate more stable MWUs for low-precision deep learning.

A useful perspective on this problem comes from neuroscience, where MWUs have close connections to synaptic dynamics (Bernstein et al., 2020; Pogodin et al., 2024; Cornford et al., 2024). Biological synapses exhibit continuous dynamic activity accompanied by noise (Choquet & Triller, 2013). Synaptic spine sizes have been observed to follow a log-normal distribution, which is thought to arise from noisy multiplicative dynamics (Loewenstein et al., 2011). In addition to activity-dependent plasticity, synaptic spines also show spontaneous fluctuations, whose dynamics are considered to be approximately proportional to their size (Loewenstein et al., 2011; Kasai et al., 2021).

These observations suggest that, to exploit the dynamics of biological synapses in artificial neural networks (ANNs), both weight updates and noise injection capturing spontaneous fluctuations should be multiplicative. However, ex-

isting ANN training methods incorporate only one of these two mechanisms. MWUs have been explored for training deep ANNs through the Madam optimizer (Bernstein et al., 2020), but they do not incorporate noise injection. In contrast, Bayesian neural networks assume a distribution over their weights and sample weights by injecting noise (Graves, 2011; Blundell et al., 2015; Khan et al., 2018). However, this noise does not inherently scale with the magnitude of each weight, and is therefore insufficient to reproduce the multiplicative fluctuations observed in biological synapses. Multiplicative noise whose magnitude scales with the weight can be interpreted as a form of data augmentation (Trinh et al., 2024), which may improve robustness to low-precision activations. Nevertheless, the corresponding weight updates are still performed using additive gradient updates (Wen et al., 2018; Bisla et al., 2022). Notably, existing work has not explicitly combined multiplicative updates, multiplicative noise injection, and log-normal weights. Zhong et al. (2022) studied log-normal weight learning but did not exploit multiplicative fluctuations.

In this paper, we propose the Log-Normal Multiplicative Dynamics (LMD) algorithm through a variational Bayesian formulation. Specifically, we use the Lie-Group Bayesian Learning Rule with a log-normal posterior over weights (Kiral et al., 2023). However, stable training can be difficult with the original formulation and it does not scale to modern deep networks. LMD addresses these limitations by (i) deriving a multiplicative weight decay regularizer and (ii) integrating a Lion-style gradient accumulation scheme (Chen et al., 2023) for multiplicative gradients. These components stabilize the learning dynamics and enable training modern deep models from scratch with low-precision forward-pass matrix multiplications, see Fig. 1.

Our contributions are as follows:

1. We achieve the first successful training of ViT and GPT-2 from scratch using MWUs. Using emulated low-precision forward passes, we demonstrate that LMD can maintain performance with low-precision computation (Fig. 1 and 4, Tables 1 and 2).
2. We show that LMD effectively suppresses the excessive weight growth that has been a major issue for standard MWU methods (Fig. 5). Through ablation studies, we verify that multiplicative weight decay strongly suppresses weight growth and provides better regularization than conventional additive weight decay (Fig. 6).
3. We show that multiplicative noise injection is important for stabilizing training dynamics when using low-precision MX data format for forward passes (Fig. 7). In these experiments, backward passes use bfloat16 (Kalamkar et al., 2019) rather than the MX data format.

2. Background and Related Work

2.1. Multiplicative Weight Updates

The unreliability of synaptic transmission lies at the heart of neural computation (Seung, 2003). The brain not only tolerates such unreliability but may even use it to drive learning (Kappel et al., 2015; Aitchison et al., 2021; Kasai et al., 2021). Synaptic transmission is driven by dynamic molecular processes within synapses, and probabilistic approaches may help to understand their robustness and plasticity (Choquet & Triller, 2013). Furthermore, spine sizes are thought to change multiplicatively (Loewenstein et al., 2011) and to store information logarithmically (Bartol et al., 2015), providing biological motivation for MWUs in ANNs (Bernstein et al., 2020; Pogodin et al., 2024; Cornford et al., 2024).

In general, MWUs attracted attention across various fields (Arora et al., 2012) such as machine learning (Littlestone, 1988; Kivinen & Warmuth, 1997), optimization (Freund & Schapire, 1997) and game theory (Grigoriadis & Khachiyan, 1995). For a loss function $\ell(\theta)$, in contrast to gradient descent (GD), MWUs modify the weights θ via multiplication, for instance, as follows:

$$\begin{aligned} \text{GD: } \theta &\leftarrow \theta - \eta \nabla \ell(\theta), \\ \text{MWU: } \theta &\leftarrow \theta \odot \exp\left(-\eta \nabla \ell(\theta) \odot \text{sign}(\theta)\right). \end{aligned} \quad (1)$$

Here, $\odot$ is an element-wise product, $\text{sign}(\theta)$ is a vector with entries in $\{-1, 0, 1\}$ depending on the sign of the entries, $\eta > 0$ is a learning rate, and exponentiation is performed element-wise. Bernstein et al. (2020) apply the above MWU to ANNs and claim that an advantage over GD is that the size of the update is proportional to the magnitude of the weight, which they claim has stabilizing properties. However, in their experiments, Madam tends to overfit, since multiplicative updates can increase the weights exponentially. They proposed weight clipping to stabilize and regularize training. However, the clipping threshold requires tuning, which motivates seeking other ways to stabilize training and avoid overfitting (Bernstein et al., 2020, Sec. 8).

2.2. Multiplicative Noise Injection

Biological synapses exhibit changes proportional to synaptic size in both activity-dependent plasticity and activity-independent spontaneous fluctuations (Loewenstein et al., 2011). The former corresponds to multiplicative update dynamics, whereas the latter can be modeled as log-normal noise injection. This view is consistent with experimental findings showing that synaptic spine sizes follow a log-normal distribution (Loewenstein et al., 2011). Although Madam’s update rule does not include a corresponding noise term (Bernstein et al., 2020), noisy fluctuations may play an important role in suppressing excessive weight growth. This motivates the use of noise injection in ANN training.

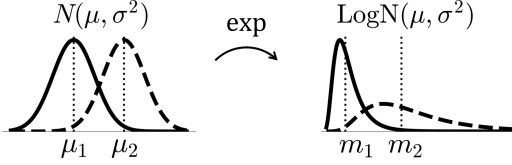


Figure 2. The figure shows a normal distribution $\mathcal{N}(\mu, \sigma^2)$ and a log-normal distribution $\text{LogN}(\mu, \sigma^2)$. In both distributions, σ is fixed and here $\mu_1 < \mu_2$. For the log-normal distribution, the median is $m = \exp(\mu)$.

Adding noise to ANN weights during training also improves generalization performance (An, 1996; Orvieto et al., 2022). Noise injection proportional to weight magnitude has recently shown promising results for generalization (Bisla et al., 2022), as well as for improved robustness to corruptions (Trinh et al., 2024). However, these studies use Gaussian noise, which is less natural from the view of multiplicative dynamics. Instead, a more natural choice is the log-normal distribution (Fig. 2):

$$q(\theta) = \text{LogN}(\theta \mid \mu, \sigma^2) = \frac{1}{\theta \sigma \sqrt{2\pi}} \exp\left(-\frac{(\log \theta - \mu)^2}{2\sigma^2}\right). \quad (2)$$

Sampling from a log-normal distribution implements a noise injection which is proportional to the median of the log-normal distribution $m = \exp(\mu)$, because if $\varepsilon \sim \text{LogN}(0, \sigma^2)$, then $\theta = m\varepsilon \sim \text{LogN}(\log m, \sigma^2)$. The standard deviation of θ is

$$\text{std}[\theta] = \sqrt{\text{Var}[\theta]} = \underbrace{me^{\frac{\sigma^2}{2}}}_{\text{mean of } \theta} \sqrt{(e^{\sigma^2} - 1)}, \quad (3)$$

which, for fixed σ , is also proportional to the mean, $\mathbb{E}_{q(\theta \mid m, \sigma^2)}[\theta] = me^{\frac{\sigma^2}{2}}$. We therefore propose using the log-normal distribution as an approximate posterior in variational learning, which we introduce next. We will see that this yields more natural multiplicative dynamics with inherent multiplicative noise injection.

2.3. Variational Learning via Lie-Group Update

Variational learning is formulated as an optimization problem over probability distributions $q(\theta)$:

$$\min_{q \in \mathcal{Q}} \mathbb{E}_{\theta \sim q}[\ell(\theta)] + \tau \mathbb{D}_{\text{KL}}(q(\theta) \parallel p_0(\theta)), \quad (4)$$

where $\ell(\theta) = \sum_{i=1}^N \ell_i(\theta)/N$ denotes the empirical risk, $p_0(\theta) \propto \exp(-R(\theta))$ is a prior, $R(\theta)$ is a regularizer, and $\tau > 0$ is a temperature parameter. $\mathcal{Q}$ denotes the set of candidate distributions, and by minimizing Eq. 4 one searches for the candidate in $\mathcal{Q}$ closest to a (tempered) Bayesian posterior, where closeness is measured by the Kullback-Leibler divergence. For $\ell_i(\theta) = -N \log p(\mathcal{D}_i \mid \theta)$ and $\tau = 1$,

this problem targets the Bayesian posterior (Zellner, 1988), where $\mathcal{D}_i$ denotes the i -th training example. Variational learning for ANNs is implemented using stochastic gradient methods, giving rise to noise injection methods (Graves, 2011; Blundell et al., 2015; Khan & Rue, 2023).

Multiplicative noise injection has previously been considered for Bayesian learning (Wen et al., 2018) but it has typically been combined with additive GD updates. Recently, multiplicative updates have been derived from a variational viewpoint by using distributions parametrized via Lie-groups (Kiral et al., 2023). We use this framework to derive a new update rule, as it naturally combines multiplicative updates with multiplicative noise injection. We start with Alg. 2 of Kiral et al. (2023, App. A.4) which optimizes Eq. 4 using the following updates:

$$\begin{aligned} \theta &\leftarrow m \odot \varepsilon \text{ and } \varepsilon \sim \text{LogN}(\varepsilon \mid 0, \sigma^2 \mathbf{I}), \\ g &\leftarrow \mathbb{E}_{q(\theta)} [\theta \odot (\nabla \ell(\theta) + \tau \nabla R(\theta)) - \tau], \\ \nu &\leftarrow \beta \nu + (1 - \beta)g, \\ m &\leftarrow m \odot \exp(-\eta \nu \odot \text{sign}(m)). \end{aligned} \quad (5)$$

Here, exponentiation operations are element-wise, and $\beta \in [0, 1)$ is a momentum parameter. From the viewpoint of Lie-groups, g and ν are tangent vectors and the $\exp$ -function is the natural mapping connecting the tangent space to the parameter space. In practice, they approximate

$$g \approx \frac{1}{S} \sum_s \theta^{(s)} \odot (\nabla \ell(\theta^{(s)}) + \tau \nabla R(\theta^{(s)})) - \tau, \quad (6)$$

where $\theta^{(s)}$ denotes the s -th Monte Carlo (MC) weight sample from $q(\theta)$ and S is the total number of MC samples. We refer to Kiral et al. (2023, Sec. 3.5) for details on the mathematical background and remark that the above method can also be viewed as gradient descent on the μ -parameter of the log-normal distribution in Eq. 2.

We specialize the updates in Kiral et al. (2023, App. A.4) to the case of $\mathcal{Q}$ being the set of log-normal distributions defined in Eq. 2 with fixed $\sigma^2 \mathbf{I}$. The parameter μ is represented through $m = \exp \mu$, where $m \in \mathbb{R}_{>0}^P$ is the median of the log-normal distribution and belongs to the multiplicative Lie-group that parametrizes the distribution. In contrast to the widely used MWU in Eq. 1, the updates in Eq. 5 (i) use noise injection, (ii) incorporate an explicit regularizer $R(\theta)$ and (iii) scale both $\nabla \ell(\theta)$ and $\nabla R(\theta)$ by the weight θ . We now clarify the last feature. Classical MWUs are derived by considering additive perturbations $\ell(\theta + t\varepsilon)$, for which the derivative at $t = 0$ yields $\nabla \ell(\theta) \odot \varepsilon$. In contrast, Kiral et al. (2023) consider multiplicative perturbations consistent with the multiplicative geometry of the parameter space. These perturbations are of the form $\ell(\theta \odot \exp(t\varepsilon)) \approx \ell(\theta \odot (1 + t\varepsilon))$ with respect to t , so that the derivative at $t = 0$ yields $\nabla \ell(\theta) \odot \theta \odot \varepsilon$. Therefore, the

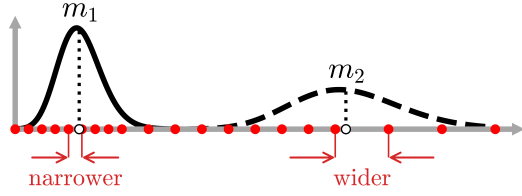


Figure 3. Log-normal distributions $\text{LogN}(\log m, \sigma^2)$ exhibit scale dependence similar to floating-point errors. Larger medians yield wider distributions, consistent with the scale-dependent growth of floating-point error. Here, $\sigma = 0.125$, and red dots show FP6 (E2M3) as an example 3-bit-mantissa format.

scaling of the gradient by the weight magnitude is a natural consequence of the type of perturbations they consider, when choosing the direction of fastest descent. Eq. 5 is the starting point for our proposed method.

A limitation of training ANNs with MWUs in Eqs. 1 and 5 is that these updates preserve the sign of each weight. When the signs are fixed, the expressive power of the network is limited (Bernstein et al., 2020, Sec. 8). To address this, we represent the weight vector as $\theta_{\text{trick}} = \theta_+ - \theta_-$ with $\theta_+, \theta_- > 0$. In the exponentiated gradient (EG) algorithm, where the sign of each weight is also preserved, this representation is known as the $\text{EG}\pm$ trick (Kivinen & Warmuth, 1997) and it has been used by Kiral et al. (2023) and Ghai et al. (2020). This $\text{EG}\pm$ trick also provides a biological interpretation consistent with Dale’s law in ANNs (Amit et al., 1989). Under this interpretation, each node is represented by separate excitatory and inhibitory neurons, see Kiral et al. (2023, Fig. 1c).

2.4. Stable Low-Precision Training via Stochastic Fluctuations

Low-precision data formats can substantially improve the throughput of general matrix multiplications (GEMMs) (Courbariaux et al., 2015; Kalamkar et al., 2019; Micikevicius et al., 2022). There have also been efforts to substantially improve power efficiency through the co-design of data formats and dedicated hardware units (Micikevicius et al., 2018; Wang et al., 2018; Ueyoshi et al., 2018; Haghi et al., 2024; Wang et al., 2025). While quantization is expected to improve the efficiency of Transformer (Vaswani et al., 2017) pre-training, quantization based pre-training often suffers from performance degradation (Chitsaz et al., 2024). To improve the efficiency of computationally intensive pre-training, learning algorithms that are robust to low-precision computation are needed. The multiplicative nature of a log-normal weight distribution (discussed in Sec. 2.2) aligns naturally with the numerical errors introduced by low-precision data formats. Rounding errors in floating-point and MX formats are roughly proportional to the magnitude of the values. Fig. 3 illustrates this relationship as an exam-

ple 3-bit-mantissa low-precision floating-point format. With fixed σ , log-normal distributions broaden as the median m increases, consistent with the wider spacing of representable values at larger magnitudes. Bernstein et al. (2020) also demonstrate the compatibility between multiplicative weight updates and low-bit logarithmic data formats. Zhao et al. (2022) point out that for additive weight updates used in logarithmic formats (Miyashita et al., 2016), the effect of rounding errors increases with weight magnitude, which leads to instability in learning.

In matrix multiplication, stochastic perturbations induced by randomized rounding can be viewed as multiplicative-like noise and have empirically been shown to stabilize low-precision training. Stochastic rounding is a technique for inducing fluctuations in the rounding process to stabilize training (Gupta et al., 2015). It randomly rounds a value to one of the nearest representable values in the lower-precision format, such that the expected value of the rounded result is equal to the original number (Crocì et al., 2022). This has been shown to enable stable low-precision training in small networks (Gupta et al., 2015; Zhang et al., 2022). Even for LLMs, training from scratch with MX data formats (Rouhani et al., 2023a) with stochastic rounding in the backward pass has been reported to retain performance (Tseng et al., 2025). Stochastic rounding has also been adopted for fine-tuning low-bit model ensembles (Nam & Lee, 2024).

Because multiplicative noise is injected before quantization, the subsequent quantization step may occasionally map a value to a representable point farther from the original value than its nearest-neighbor quantization point (Fig. 3). Considering that nearest-neighbor rounding is not always optimal in quantization (Nagel et al., 2020), our results suggest that the stochasticity introduced by this noise injection can stabilize learning with low-precision formats, possibly via a mechanism analogous to stochastic rounding.

Microscaling (MX) Data Formats Microscaling (MX) data formats (Rouhani et al., 2023a) are one family of the proposed formats in Rouhani et al. (2023b) with the aim of achieving wide dynamic range with limited bit width. In MX data formats, a collection of k_{mx} numbers is represented jointly, using very low precision private parts p_{mx} and a shared integer exponent s_{mx} . In the standard MX settings, $k_{\text{mx}} = 32$, and s_{mx} is an INT8 shared exponent. Each private element p_{mx} represents a number of the form $2^{s_{\text{mx}} - b_{\text{mx}}} p_{\text{mx}}$ where b_{mx} is called a bias. For example, in the MXFP6 and MXFP4 data formats the bias term is 1 and the private elements p_{mx} either use the FP6 (E2M3) representation with 1-bit sign, 2-bit exponent and 3-bit mantissa, or use the FP4 (E2M1) representation with 1-bit sign, 2-bit exponent, and 1-bit mantissa. MX data formats have also been considered for deployment of LLMs because they enable

Algorithm 1 Proposed Log-Normal Multiplicative Dynamics (LMD) Optimizer

Require: Learning rate $\eta > 0$, temperature $\tau > 0$, $\beta_1, \beta_2 \in [0, 1]$, log-normal prior $m_r > 0$, log-normal std. $\sigma > 0$

- 1: Initialize $\mathbf{m} = \begin{bmatrix} \mathbf{m}_+ \\ \mathbf{m}_- \end{bmatrix} \in \mathbb{R}_{>0}^{2P}$ as described in Sec. 3.2, $\boldsymbol{\nu} = \begin{bmatrix} \boldsymbol{\nu}_+ \\ \boldsymbol{\nu}_- \end{bmatrix} = \mathbf{0} \in \mathbb{R}^{2P}$ and $\mathbf{A}^\top = \begin{bmatrix} \mathbf{I} \\ -\mathbf{I} \end{bmatrix} \in \mathbb{R}^{2P \times P}$
- 2: **while** not converged **do**
- 3: $\boldsymbol{\varepsilon} \sim \text{LogN}(0, \sigma^2 \mathbf{I}_{2P})$ # sample log-normal noise
- 4: $\boldsymbol{\theta} \leftarrow \mathbf{m} \odot \boldsymbol{\varepsilon}$ # multiplicative noise injection
- 5: $\mathbf{g} \leftarrow \boldsymbol{\theta} \odot \mathbf{A}^\top \hat{\nabla} \ell(\boldsymbol{\theta}_{\text{trick}})$ where $\boldsymbol{\theta}_{\text{trick}} = \mathbf{A} \boldsymbol{\theta}$ # compute stochastic gradient of loss using EG $\pm$ trick
- 6: $\mathbf{r} \leftarrow \tau(\boldsymbol{\theta} \odot \nabla R_{m_r}(\boldsymbol{\theta}) - 1)$ # compute regularization gradient
- 7: $\boldsymbol{\nu} \leftarrow \beta_2 \boldsymbol{\nu} + (1 - \beta_2) \mathbf{g}$ and $\boldsymbol{\nu}_{\text{temp}} \leftarrow \beta_1 \boldsymbol{\nu} + (1 - \beta_1) \mathbf{g}$ # update momentum
- 8: $\mathbf{m} \leftarrow \mathbf{m} \odot \exp(-\eta(\text{sign}(\boldsymbol{\nu}_{\text{temp}}) + \mathbf{r}))$ # multiplicative update of median
- 9: **end while**

improved throughput of matrix multiplications (Rouhani et al., 2023c; Tseng et al., 2025). In addition, adopting the MX data format for deep learning models will contribute to the development of future energy-efficient hardware. For example, an MXFP6-based computing system is estimated to provide approximately a 2 $\times$ advantage in hardware cost compared to FP8 (Rouhani et al., 2023b).

3. Log-Normal Multiplicative Dynamics

We extend Eq. 5 by incorporating a design similar to synaptic multiplicative dynamics, enabling stable low-precision training in large networks and overcoming the small-scale limitation of Kiral et al. (2023).

Alg. 1 shows the LMD Optimizer (the main differences from Eq. 5 are highlighted in red). As Loshchilov & Hutter (2019) show, incorporating gradient and weight penalties with momentum accumulation tends to destabilize learning and degrade performance. This instability arises because, when either the gradient or the penalty dominates, the accumulated moment neglects the other term (Bjorck et al., 2021). LMD addresses this issue by decoupling the gradient $\mathbf{g}$ and the regularizer $\mathbf{r}$ as in steps 5 and 6 of Alg. 1, as used in AdamW. We also use signed gradient momentum (Bernstein et al., 2018) and independent weight penalties, alongside two momentum coefficients, β_1 and β_2 , as in Lion (Chen et al., 2023). The default exponential moving average (EMA) coefficient for momentum $\boldsymbol{\nu}$ is $\beta_2 = 0.99$ in step 7 of Alg. 1. In each update, we interpolate between the current gradient $\mathbf{g}$ and the momentum $\boldsymbol{\nu}$ using $\beta_1 = 0.95$, and call that interpolation $\boldsymbol{\nu}_{\text{temp}}$. We then apply a sign operation to yield the update rule in step 8 of Alg. 1:

$$\mathbf{m} \leftarrow \mathbf{m} \odot \exp(-\eta(\text{sign}(\boldsymbol{\nu}_{\text{temp}}) + \mathbf{r})). \quad (7)$$

In the Lie group framework, past gradients can be accumulated because all the update directions lie in the same vector space (Kiral et al., 2023, Sec. 3.5). The method equally allows us to update with the component-wise sign of the momentum. We assume a component-wise regularizer $\tilde{R}$ defined below in Sec. 3.1, so that $R(\boldsymbol{\theta}) = \sum_{i=1}^{2P} \tilde{R}(\theta_i)$ and

we set $\tau^{-1} := \tilde{R}'(1) - 1$ so that the individual entries of $\mathbf{r}$ are $r_i = 1$ when the entries of $\boldsymbol{\theta}$ are $\theta_i = 1$. Consequently, when $\theta_i > 1$, the regularization term exceeds the component-wise signed update term $\text{sign}(\nu_{i,\text{temp}})$, preventing the corresponding individual entry m_i from increasing further. This is also related to stability. From Eq. 3, when σ is fixed, the standard deviation of the multiplicative noise is proportional to the magnitude of the mean weight. Therefore, larger weights also imply larger noise, which increases the risk of unstable learning. The soft clipping effect improves stability by suppressing excessive weight increases. We only need to maintain two variables, $\mathbf{m}$ and $\boldsymbol{\nu}$. Then, P weight parameters require additional P variables. The EG $\pm$ trick doubles the weights, so LMD holds $4P$ parameters in total. By contrast, AdamW keeps P weights plus $2P$ momentum terms for a total of $3P$, meaning LMD uses one additional P -dimensional vector compared to AdamW.

LMD also uses MC sampling and can efficiently take multiple MC samples, which effectively reduce variance (Kingma et al., 2015), in multi-GPU environments. When using multiple MC sampling, $\mathbf{g}$ and $\mathbf{r}$ as in steps 5 and 6 of Alg. 1 are redefined as:

$$\begin{aligned} \mathbf{g} &\leftarrow \frac{1}{JS} \sum_{j,s} \boldsymbol{\theta}_j^{(s)} \odot \mathbf{A}^\top \hat{\nabla} \ell(\mathbf{A} \boldsymbol{\theta}_j^{(s)}), \\ \mathbf{r} &\leftarrow \frac{1}{JS} \sum_{j,s} \tau(\boldsymbol{\theta}_j^{(s)} \odot \nabla R(\boldsymbol{\theta}_j^{(s)}) - 1), \end{aligned} \quad (8)$$

where J is the number of devices and S is the number of MC samples per device. Here, $\hat{\nabla} \ell$ denotes a stochastic gradient. We draw a different random weight sample $\boldsymbol{\theta}_j^{(s)}$ for each device j and each MC sample s . LMD can be implemented as a drop-in replacement for Adam, similar to IVON (Shen et al., 2024). See App. A for more details.

3.1. Multiplicative Weight Decay

In LMD, we incorporate weight decay by placing a log-normal prior in the variational objective in Eq. 4. We choose

a component-wise log-normal prior:

$$\begin{aligned}\tilde{R}_{m_r}(\theta) &= -\log p_0(\theta) \\ &= -\log(\text{LogN}(\theta \mid \log m_r, \sigma^2)),\end{aligned}\quad (9)$$

where $R_{m_r}(\theta) = \sum_{i=1}^{2P} \tilde{R}_{m_r}(\theta_i)$. For a single component θ , the corresponding component r of the regularization gradient $\mathbf{r}$ used in Eq. 7 is given by:

$$\begin{aligned}r &= \tau(\theta \tilde{R}'_{m_r}(\theta) - 1) \\ &= \tau \left[\theta \cdot \frac{1}{\theta} \left(1 + \frac{\log \theta - \log m_r}{\sigma^2} \right) - 1 \right] \\ &= \tau \left(\frac{\log \theta - \log m_r}{\sigma^2} \right).\end{aligned}\quad (10)$$

To understand the effect of this log-normal prior on the learning dynamics, consider the expected regularization step $-\eta r$. Defining $\alpha = \frac{\eta \tau}{\sigma^2} \geq 0$, we have

$$\mathbb{E}[-\eta r] = \mathbb{E} \left[-\alpha \log \left(\frac{\theta}{m_r} \right) \right] = \log \left(\frac{m}{m_r} \right)^{-\alpha}, \quad (11)$$

where we used the fact that $\log \theta$ is normally distributed with mean $\log m$. Thus, the LMD update can be written as:

$$\mathbf{m} \leftarrow (\mathbf{m}^{1-\alpha} \mathbf{m}_r^\alpha) \odot \exp(-\eta \text{sign}(\nu_{\text{temp}})), \quad (12)$$

which is equivalent to

$$\log \mathbf{m} \leftarrow (1-\alpha) \log \mathbf{m} + \alpha \log \mathbf{m}_r - \eta \text{sign}(\nu_{\text{temp}}). \quad (13)$$

As shown in Eq. 13, this simply corresponds to weight decay in logarithmic space. Unlike standard additive weight decay, this penalty drives m toward m_r rather than zero. When used with the $\text{EG}\pm$ trick, the positive and negative components are both driven toward m_r , so that the expected value of $\theta_{\text{trick}} = \theta_+ - \theta_-$ becomes zero. Larger m_r increase the strength of multiplicative noise injection, while θ_{trick} preserves zero-mean in expectation.

As discussed by Trinh et al. (2024), multiplicative weight noise can be interpreted as activation fluctuations and hence as a form of data augmentation. For each term in the preactivation of the j -th neuron in a linear layer, we have:

$$\underbrace{m_{ji}(1 + \sigma \varepsilon_{ji})}_{\text{perturbed weight}} x_i = m_{ji} \underbrace{(1 + \sigma \varepsilon_{ji})}_{\text{perturbed activation}} x_i, \quad (14)$$

where x_i is the input activation and $\varepsilon_{ji} \sim \mathcal{N}(0, 1)$ is the injected Gaussian noise. In LMD, the log-normal perturbation plays an analogous role. Due to the $\text{EG}\pm$ trick, it can still induce activation fluctuations, even when the expected value of θ_{trick} is zero, rather than simply pruning the connection. Similar to Trinh et al. (2024), we conjecture activation fluctuations to be important for generalization.

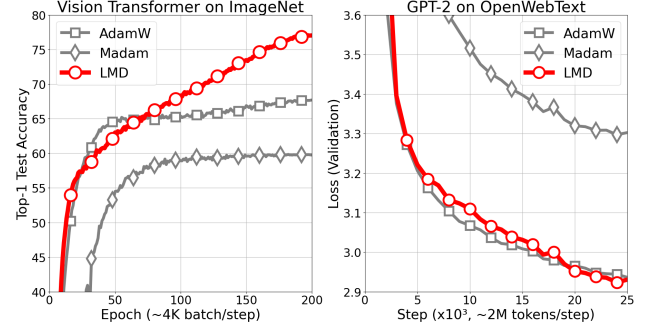


Figure 4. LMD achieves performance comparable to AdamW on GPT-2 and outperforms it on ViT when trained with bfloat16 forward passes. Madam fails to train adequately even when trained in bfloat16. The experimental setup matches that of Fig. 1, except that the forward passes now use bfloat16 instead of MXFP6.

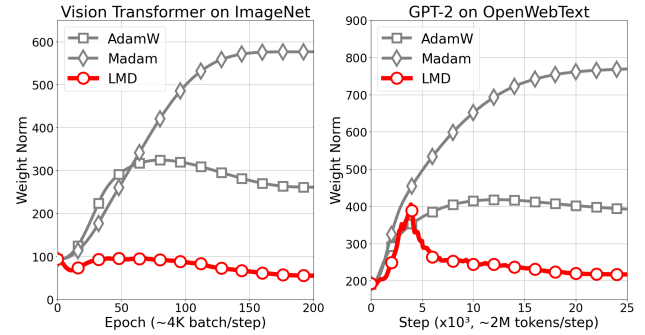


Figure 5. The dynamics of weight ℓ_2 -norm during training of the models shown in Fig. 4 using bfloat16. Madam exhibits a large weight norm, while LMD converges to a norm close to the initial value, suppressing excessive weight increase.

3.2. Initialization

We initialize the LMD median parameters m_+ and m_- by choosing their initial values $m_{+,0}$ and $m_{-,0}$ so that

$$\theta_0 = \mathbb{E}_{q(\theta_+ | m_{+,0}, \sigma^2)}[\theta_+] - \mathbb{E}_{q(\theta_- | m_{-,0}, \sigma^2)}[\theta_-], \quad (15)$$

where θ_0 denotes the default initialization value for the corresponding neural network parameter. To satisfy Eq. 15, we set the corresponding entries of $\mathbf{m}$ using m_r as follows:

$$\begin{aligned}m_{+,0} &= \begin{cases} \theta_0 \exp(-\frac{\sigma^2}{2}) + m_r, & \theta_0 > 0, \\ m_r, & \theta_0 \leq 0, \end{cases} \\ m_{-,0} &= \begin{cases} m_r, & \theta_0 > 0, \\ -\theta_0 \exp(-\frac{\sigma^2}{2}) + m_r, & \theta_0 \leq 0. \end{cases}\end{aligned}\quad (16)$$

Entries that do not have the signed offset from θ_0 are set to m_r , for which the multiplicative decay factor equals 1. This construction lets us reuse the value θ_0 from standard deep learning initialization schemes.

For parameters whose default initialization is one, such as scale parameters in normalization layers, we instead set

Table 1. ViT performance comparison; mean $\pm$ std. (n = 3 runs). LMD uses sampled weight values during training and the expected value of the weight during testing.

Dataset/ Architecture	Optimizer	Forward Precision	Train Accuracy (%) Top1 / Top5	Test Accuracy (%) Top1 / Top5	Weight Norm
ImageNet/ Vision Transformer	AdamW	bfloat16	93.67 $\pm$ 0.06/97.83 $\pm$ 0.03	68.11 $\pm$ 0.38/86.63 $\pm$ 0.21	260.7 $\pm$ 0.5
		MXFP6	93.41 $\pm$ 0.12/97.73 $\pm$ 0.04	67.98 $\pm$ 0.27/86.54 $\pm$ 0.15	261.5 $\pm$ 0.4
	Madam	bfloat16	84.36 $\pm$ 0.03/94.27 $\pm$ 0.02	60.14 $\pm$ 0.31/81.45 $\pm$ 0.18	577.3 $\pm$ 0.9
		MXFP6	12.11 $\pm$ 0.17/27.88 $\pm$ 0.30	13.47 $\pm$ 0.16/30.59 $\pm$ 0.27	72.6 $\pm$ 0.4
	LMD (Proposed)	bfloat16	79.84 $\pm$ 0.16/93.53 $\pm$ 0.06	77.06$\pm$0.08/93.35$\pm$0.04	55.2 $\pm$ 0.1
		MXFP6	79.74 $\pm$ 0.34/93.50 $\pm$ 0.14	77.15$\pm$0.08/93.42$\pm$0.05	55.5 $\pm$ 0.3

Table 2. GPT-2 performance comparison; mean $\pm$ std. (n = 3 runs). LMD uses the expected value of the weight during validation.

Dataset/ Architecture	Optimizer	Forward Precision	sequence length 1024		sequence length 4096	
			Val. Loss	Weight Norm	Val. Loss	Weight Norm
OpenWebText/ GPT-2	AdamW	bfloat16	2.937 $\pm$ 0.000	392.7 $\pm$ 0.4	4.791 $\pm$ 2.018	931.7 $\pm$ 70.5
		MXFP6	3.015 $\pm$ 0.000	424.9 $\pm$ 0.3	2.980 $\pm$ 0.002	426.1 $\pm$ 0.2
	Madam	bfloat16	3.315 $\pm$ 0.001	762.4 $\pm$ 0.6	3.303 $\pm$ 0.003	765.1 $\pm$ 3.4
		MXFP6	4.620 $\pm$ 0.006	212.4 $\pm$ 1.3	4.575 $\pm$ 0.015	221.2 $\pm$ 1.5
	LMD (Proposed)	bfloat16	2.961 $\pm$ 0.002	217.2 $\pm$ 4.5	2.926$\pm$0.006	212.9 $\pm$ 2.1
		MXFP6	2.971 $\pm$ 0.001	215.3 $\pm$ 2.9	2.927$\pm$0.002	216.9 $\pm$ 0.9

$m_{+,0}^{\text{scale}} = \exp(-\sigma^2/2)$ and remove the negative component. Also, for these parameters, we set $m_{\tau}^{\text{scale}} = \exp(-\sigma^2/2)$, $(\tau^{\text{scale}})^{-1} = 2R'(2) - 1$. This choice sets the mean of the scale parameter to 1 and sets the soft-clipping point to 2. Although heuristic, this choice worked well in practice and stabilized training.

4. Experiments

We first compare AdamW, Madam and LMD when training ViT and GPT-2 from scratch in Sec. 4.1. We then validate the effectiveness of the multiplicative weight-decay regularizer derived in Eq. 12 (Sec. 4.2). Finally, we assess the role of multiplicative noise injection in enabling low-precision training (Sec. 4.3). All experiments with the MX format were performed via software emulation. See App. B for detailed experimental settings. In addition, App. C compares LMD with the original algorithm proposed by Kiral et al. (2023) and compares LMD with Madam in a setting where Madam performs well. In both cases, LMD achieves superior performance. App. D reports results obtained without the EG $\pm$ trick. App. F provides a detailed analysis of memory usage.

4.1. Comparison with Existing Optimizers

We compare LMD with AdamW and Madam (Fig. 1, 4 and 5). The detailed results are shown in Tables 1 and 2. In ViT training, LMD substantially outperforms both AdamW and Madam (Table 1). In GPT-2 training with a maximum

sequence length of 4096, LMD achieves better performance than all AdamW variants. In MXFP6 forward-pass training, AdamW often performs poorly or produces inconsistent results, whereas LMD remains stable. These results show that LMD can effectively leverage low-precision forward computations while maintaining stable and accurate training. Madam performs worse in all settings. As shown in Fig. 5, its weight ℓ_2 -norm tends to grow relative to the other optimizers, indicating unstable training. By contrast, the weight ℓ_2 -norm of LMD remains close to its initial value, suggesting that LMD seeks optimal solutions in regions of parameter space with relatively small weight norms. For LMD, the weight norms are computed using the mean value of $\theta_+ - \theta_-$. Note that in Fig. 5, the initial weight norm under LMD is the same as under AdamW and Madam, due to the initialization described in Sec. 3.2.

4.2. Effectiveness of Multiplicative Weight Decay

We conduct an ablation study to understand why LMD is effective. The regularizing effect of LMD arises not only from the multiplicative weight update in Eq. 1 but also from scaling the gradient by the weight, as shown in Eq. 5 and from multiplicative weight decay, as shown in Eq. 12. We show that these two features suppress the excessive weight growth characteristic of multiplicative updates. Fig. 6 shows four variants: multiplicative weight decay or additive weight decay (i.e. Gaussian prior), and each of these two cases with updates by weight-scaled gradient $\theta \odot \nabla \ell(\theta)$ or just $\nabla \ell(\theta)$. Since the observed trends for the momentum ν are em-

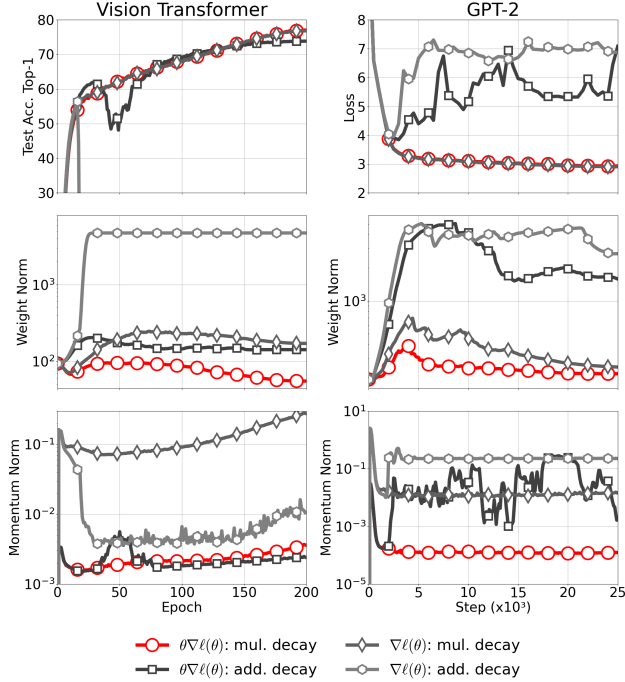


Figure 6. The effectiveness of multiplicative weight decay in LMD. By scaling the gradient according to the weight magnitude and employing multiplicative weight decay in LMD, we observed consistently stable training dynamics in both ViT and GPT-2.

pirically insensitive to the sign of the weights, we plot the ℓ_2 -norm of the momentum for positive weights. The momentum norm captures the dynamics of the gradient magnitudes through accumulation of past gradients. Additive weight decay was evaluated by setting $r = (\theta - m)/(1 - m_r)$ in Eq. 10. This choice ensures that when θ is one, r is also one. Overall, multiplicative weight decay stabilizes LMD training more effectively than additive decay. Under additive decay, the weight ℓ_2 -norm tends to increase, indicating weak regularization, and this effect is especially pronounced in GPT-2. Under multiplicative decay, when the gradient is scaled by the weight, fluctuations in the weight ℓ_2 -norm are more gradual than in the unscaled case, demonstrating effective regularization. The momentum norm is also smaller in magnitude, indicating suppressed gradient norm.

4.3. Effectiveness of Multiplicative Noise for Low-Precision Training

We investigate the role of multiplicative noise injection in low-precision training using emulated computation with the MX data format. The key question is whether low-precision training follows dynamics similar to those of high-precision training. Even when the final performance is comparable, a clear difference in weight norms suggests convergence to a different region of the parameter space. Multiplicative weight noise suppresses the growth of the momentum norm

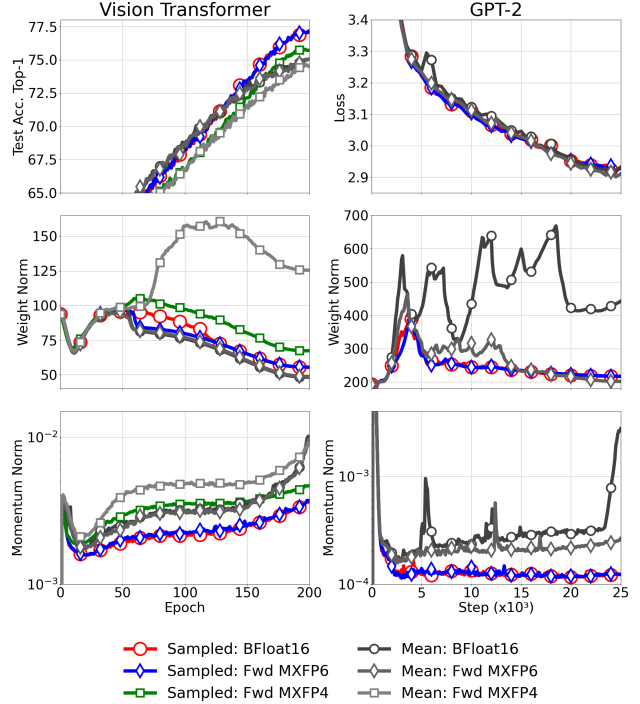


Figure 7. The effectiveness of multiplicative noise injection for low-precision training. Sampling weights enables weight regularization and gradient stabilization.

and yields consistent learning dynamics across precision settings, with or without MXFP6 forward passes. Fig. 7 compares four settings for GPT-2: training with bfloat16 forward passes and MXFP6 forward passes, each combined with either sampled training or mean training. For ViT, we additionally include MXFP4 results, resulting in six settings. In the ViT experiments, sampled training clearly outperforms mean training in terms of accuracy. We attribute this improvement to the fact that, in the final phase of training (when cosine annealing has a strong effect), sampling better suppresses the growth of the momentum ℓ_2 -norm, which helps prevent overfitting. Looking at the MXFP4 results, the mean training (black) exhibits a noticeably larger weight ℓ_2 -norm than sampled training (green), indicating that sampling is an important regularizer in low-precision training. In the GPT-2 experiments, sampled training shows almost identical dynamics regardless of precision, demonstrating that LMD’s multiplicative noise injection enables stable low-precision training. Conversely, mean training with MXFP6 forward passes shows a clear increase in weight ℓ_2 -norm, indicating insufficient regularization. Nevertheless, the final performance of the mean training using MXFP6 was close to that of sampled training. This suggests that the numerical error in MXFP6 may act as an additional source of regularization, although the resulting dynamics differ from those observed with bfloat16 forward passes.

5. Discussion

This work is biologically motivated and asks whether multiplicative weight updates are feasible for low-precision deep learning. To evaluate optimizer stability, we focus on training of ViT and GPT-2 from scratch. Although we demonstrated stable training when using MXFP6 for forward passes, we have not evaluated low-precision backward passes, and because all experiments were conducted via emulation, we did not measure real-world speedups on hardware that natively supports MX data formats, as in Rouhani et al. (2023c). In our preliminary experiments with LMD, low-precision backward passes did not perform well. One possible reason is that LMD applies (multiplicative) stochastic perturbations to the weights, but does not take the quantization error in the backward pass into account. On the other hand, low-precision backward passes can be made effective using stochastic rounding (SR) (Tseng et al., 2025), so combining LMD and SR is a promising direction. Low-precision forward passes using SR tend to diverge when used for training (NVIDIA et al., 2025, Fig 10), suggesting that LMD may play a complementary role in this setting. The EG $\pm$ trick doubles the number of weight parameters that need to be learned, which increase memory usage. However, in large transformers such as GPT, the memory usage bottleneck lies in the activations, not in the weight parameters themselves (Chitsaz et al., 2024; Chen et al., 2025). While the EG $\pm$ trick doubles the number of weights, the number of weights used in matrix multiplication remains unchanged, so it does not affect activation memory usage or the computational cost of GEMMs. We have not investigated LMD for fine-tuning and other downstream tasks. We believe that our promising results for LMD will motivate further research in these areas.

Conclusions We propose LMD, a multiplicative weight update (MWU) algorithm that incorporates the log-normal multiplicative dynamics observed in biological synapses. The multiplicative regularization introduced by LMD suppresses the excessive weight growth that was a challenge for MWUs, making it possible to train large-scale neural networks such as ViT and GPT-2. Notably, even when using the low-precision MXFP6 data format for the forward pass, no performance degradation was observed. These results suggest that LMD has the potential to serve as an effective optimizer when used with energy-efficient hardware architectures designed for low-precision computation.

Acknowledgements

This work was supported by JST CREST Grant Number JPMJCR2112, JST Research and Development Program for Next-generation Edge AI Semiconductors Grant Number JPMJES2512, JSPS KAKENHI Grant Number JP26K14955

and the RIKEN Special Postdoctoral Researcher Program. Computational resources were provided by the TSUBAME 4.0 supercomputer at the Institute of Science Tokyo. We thank Bai Cong for setting up support for the GPT-2 experiments, and Christopher J. Anders for feedback on a draft of this paper.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Aitchison, L., Jegminat, J., Menendez, J. A., Pfister, J.-P., Pouget, A., and Latham, P. E. Synaptic plasticity as Bayesian inference. *Nature Neuroscience*, 24(4):565–571, 2021.
- Amit, D. J., Wong, K. Y. M., and Campbell, C. Perceptron learning with sign-constrained weights. *Journal of Physics A: Mathematical and General*, 22(12):2039, 1989.
- An, G. The Effects of Adding Noise During Backpropagation Training on a Generalization Performance. *Neural Computation*, 8(3):643–674, 1996.
- Arora, S., Hazan, E., and Kale, S. The Multiplicative Weights Update Method: a Meta-Algorithm and Applications. *Theory of Computing*, 8(6):121–164, 2012.
- Bartol, T. M., Bromer, C., Kinney, J., Chirillo, M. A., Bourne, J. N., Harris, K. M., and Sejnowski, T. J. Nanoconnectomic upper bound on the variability of synaptic plasticity. *eLife*, 4:e10778, 2015.
- Bernstein, J., Wang, Y., Aizzadenesheli, K., and Anandkumar, A. signSGD: compressed optimisation for non-convex problems. In *International Conference on Machine Learning*, 2018.
- Bernstein, J., Zhao, J., Meister, M., Liu, M.-Y., Anandkumar, A., and Yue, Y. Learning compositional functions via multiplicative weight updates. In *Advances in Neural Information Processing Systems*, 2020.
- Bisla, D., Wang, J., and Choromanska, A. Low-Pass Filtering SGD for Recovering Flat Optima in the Deep Learning Optimization Landscape. In *International Conference on Artificial Intelligence and Statistics*, 2022.
- Bjorck, J., Weinberger, K., and Gomes, C. Understanding Decoupled and Early Weight Decay. In *AAAI Conference on Artificial Intelligence*, 2021.

- Blundell, C., Cornebise, J., Kavukcuoglu, K., and Wierstra, D. Weight Uncertainty in Neural Networks. In *International Conference on Machine Learning*, 2015.
- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., and Le, Q. V. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023.
- Chen, Y., Zhang, Y., Liu, Y., Yuan, K., and Wen, Z. A memory efficient randomized subspace optimization method for training large language models. In *International Conference on Machine Learning*, 2025.
- Chitsaz, K., Fournier, Q., Mordido, G., and Chandar, S. Exploring Quantization for Efficient Pre-Training of Transformer Language Models. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 13473–13487, 2024.
- Choquet, D. and Triller, A. The Dynamic Synapse. *Neuron*, 80(3):691–703, 2013.
- Cornford, J., Pogodin, R., Ghosh, A., Sheng, K., Bicknell, B. A., Codol, O., Clark, B. A., Lajoie, G., and Richards, B. A. Brain-like learning with exponentiated gradients. *bioRxiv*, 2024.
- Courbariaux, M., Bengio, Y., and David, J.-P. Training deep neural networks with low precision multiplications. In *Workshop Track Proceedings of the International Conference on Learning Representations*, 2015.
- Croci, M., Fasi, M., Higham, N. J., Mary, T., and Mikaitis, M. Stochastic rounding: implementation, error analysis and applications. *Royal Society Open Science*, 9:211631, 2022.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*, 2021.
- Freund, Y. and Schapire, R. E. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- Ghai, U., Hazan, E., and Singer, Y. Exponentiated Gradient Meets Gradient Descent. In *International Conference on Algorithmic Learning Theory*, 2020.
- Gokaslan, A. and Cohen, V. Openwebtext corpus. <http://Skylion007.github.io/OpenWebTextCorpus>, 2019.
- Graves, A. Practical Variational Inference for Neural Networks. In *Advances in Neural Information Processing Systems*, volume 24, 2011.
- Grigoriadis, M. D. and Khachiyan, L. G. A sublinear-time randomized approximation algorithm for matrix games. *Oper. Res. Lett.*, 18(2):53–58, 1995.
- Gupta, S., Agrawal, A., Gopalakrishnan, K., and Narayanan, P. Deep learning with limited numerical precision. In *International Conference on Machine Learning*, 2015.
- Haghi, P., Wu, C., Azad, Z., Li, Y., Gui, A., Hao, Y., Li, A., and Geng, T. T. Bridging the Gap Between LLMs and LNS with Dynamic Data Format and Architecture Codesign. In *IEEE/ACM International Symposium on Microarchitecture*, 2024.
- Kalamkar, D., Mudigere, D., Mellempudi, N., Das, D., Banerjee, K., Avancha, S., Vooturi, D. T., Jammalamadaka, N., Huang, J., Yuen, H., Yang, J., Park, J., Heinecke, A., Georganas, E., Srinivasan, S., Kundu, A., Smelyanskiy, M., Kaul, B., and Dubey, P. A study of bfloat16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019.
- Kappel, D., Habenschuss, S., Legenstein, R., and Maass, W. Network Plasticity as Bayesian Inference. *PLOS Computational Biology*, 11(11):e1004485, 2015.
- Karpathy, A. NanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- Kasai, H., Ziv, N. E., Okazaki, H., Yagishita, S., and Toyozumi, T. Spine dynamics in the brain, mental disorders and artificial neural networks. *Nature Reviews Neuroscience*, 22:407–422, 2021.
- Khan, M. E. and Rue, H. The Bayesian Learning Rule. *Journal of Machine Learning Research*, 24(281):1–46, 2023.
- Khan, M. E., Nielsen, D., Tangkaratt, V., Lin, W., Gal, Y., and Srivastava, A. Fast and Scalable Bayesian Deep Learning by Weight-Perturbation in Adam. In *International Conference on Machine Learning*, 2018.
- Kingma, D. P., Salimans, T., and Welling, M. Variational dropout and the local reparameterization trick. In *Advances in Neural Information Processing Systems*, 2015.
- Kiral, E. M., Moellenhoff, T., and Khan, M. E. The Lie-Group Bayesian Learning Rule. In *International Conference on Artificial Intelligence and Statistics*, 2023.

- Kivinen, J. and Warmuth, M. K. Exponentiated Gradient versus Gradient Descent for Linear Predictors. *Information and Computation*, 132(1):1–63, 1997.
- Littlestone, N. Learning Quickly When Irrelevant Attributes Abound: A New Linear-Threshold Algorithm. *Mach. Learn.*, 2(4):285–318, 1988.
- Loewenstein, Y., Kuras, A., and Rumpel, S. Multiplicative Dynamics Underlie the Emergence of the Log-Normal Distribution of Spine sizes in the Neocortex In Vivo. *Journal of Neuroscience*, 31(26):9481–9488, 2011.
- Loshchilov, I. and Hutter, F. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019.
- Micikevicius, P., Narang, S., Alben, J., Diamos, G. F., Elsen, E., García, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. Mixed Precision Training. In *International Conference on Learning Representations*, 2018.
- Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., Mellempudi, N., Oberman, S., Shoeybi, M., Siu, M., and Wu, H. FP8 Formats for Deep Learning. *arXiv preprint arXiv:2209.05433*, 2022.
- Miyashita, D., Lee, E. H., and Murmann, B. Convolutional Neural Networks using Logarithmic Data Representation. *arXiv preprint arXiv:1603.01025*, 2016.
- Nagel, M., Amjad, R. A., Van Baalen, M., Louizos, C., and Blankevoort, T. Up or Down? Adaptive Rounding for Post-Training Quantization. In *International Conference on Machine Learning*, 2020.
- Nam, G. and Lee, J. Ex Uno Pluria: Insights on Ensembling in Low Precision Number Systems. In *Advances in Neural Information Processing Systems*, 2024.
- NVIDIA, Abecassis, F., Agrusa, A., Ahn, D., Alben, J., Alborghetti, S., Andersch, M., Arayandi, S., Bjorlin, A., Blakeman, A., Briones, E., Buck, I., Catanzaro, B., Chang, M., Choi, J., Chrzanowski, M., Chung, E., Cui, V., Dai, S., Rouhani, B. D., del Mundo, C., Donia, D., Eryilmaz, B., Estela, H., Goel, A., Goncharov, O., Guvvala, Y., Hesse, R., Hewett, R., Hum, H., Kapasi, U., Khailany, B., Khona, M., Knight, N., Kondratenko, A., Krashinsky, R., Lanir, B., Layton, S., Lightstone, M., Lo, D., Micikevicius, P., Mishra, A., Moon, T., Narayanan, D., Ni, C., Paithankar, A., Pasumarthi, S., Patel, A., Patwary, M., Poojary, A., Prasad, G., Priyadarshi, S., Qin, Y., Ren, X., Rybakov, O., Sakr, C., Satheesh, S., Sergienko, S., Shamis, P., Shankar, K., Sharma, N., Shoeybi, M., Siu, M., Smelyanskiy, M., Stosic, D., Stosic, D., Su, B.-Y., Sun, F., Tajbakhsh, N., Thomas, S., Tredak, P., Tsykunov, E., Vaithilingam, G., Vavre, A., Venkatesan, R., Waleffe, R., Wan, Q., Wang, H., Wang, M., Wei, L., Wu, H., Wu, E., Wyss, K., Xu, N., Xue, J., Yang, C., Zhai, Y., Zhang, R., Zhu, J., and Zhu, Z. Pretraining large language models with nvfp4. *arXiv preprint*, 2025.
- Orvieto, A., Kersting, H., Proske, F., Bach, F., and Lucchi, A. Anticorrelated Noise Injection for Improved Generalization. In *International Conference on Machine Learning*, 2022.
- Pogodin, R., Cornford, J., Ghosh, A., Gidel, G., Lajoie, G., and Richards, B. Synaptic weight distributions depend on the geometry of plasticity. In *International Conference on Learning Representations*, 2024.
- Project. MX Pytorch Emulation Library. <https://github.com/microsoft/microxcaling>, 2023.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language Models are Unsupervised Multi-task Learners. *OpenAI blog*, 2019.
- Rouhani, B., Garegrat, N., Savell, T., More, A., Han, K.-N., Zhao, M., Hall, R., Klar, J., Chung, E., Yu, Y., Schulte, M., Wittig, R., Bratt, I., Stephens, N., Milanovic, J., Brothers, J., Dubey, P., Cornea, M., Heinecke, A., Rodriguez, A., Langhammer, M., Deng, S., Naumov, M., Micikevicius, P., Siu, M., and Verrilli, C. OCP Microscaling (MX) Specification, Version 1.0. Specification, Open Compute Project, 2023a.
- Rouhani, B., Zhao, R., Elango, V., Shafipour, R., Hall, M., Mesmakhosroshahi, M., More, A., Melnick, L., Golub, M., Varatkar, G., Shao, L., Kolhe, G., Melts, D., Klar, J., L’Heureux, R., Perry, M., Burger, D., Chung, E., Deng, Z. S., Naghshineh, S., Park, J., and Naumov, M. With Shared Microexponents, A Little Shifting Goes a Long Way. In *International Symposium on Computer Architecture*, 2023b.
- Rouhani, B., Zhao, R., More, A., Hall, M., Khodamoradi, A., Deng, S., Choudhary, D., Cornea, M., Dellinger, E., Denolf, K., Dusan, S., Elango, V., Golub, M., Heinecke, A., James-Roxby, P., Jani, D., Kolhe, G., Langhammer, M., Li, A., Melnick, L., Mesmakhosroshahi, M., Rodriguez, A., Schulte, M., Shafipour, R., Shao, L., Siu, M., Dubey, P., Micikevicius, P., Naumov, M., Verrilli, C., Wittig, R., Burger, D., and Chung, E. Microscaling Data Formats for Deep Learning. *arXiv preprint arXiv:2310.10537*, 2023c.
- Seung, H. Learning in Spiking Neural Networks by Reinforcement of Stochastic Synaptic Transmission. *Neuron*, 40(6):1063–1073, 2003.

- Shen, Y., Daheim, N., Cong, B., Nickl, P., Marconi, G. M., Bazan, C., Yokota, R., Gurevych, I., Cremers, D., Khan, M. E., and Möllenhoff, T. Variational Learning is Effective for Large Deep Networks. In *International Conference on Machine Learning*, 2024.
- Trinh, T., Heinonen, M., Acerbi, L., and Kaski, S. Improving robustness to corruptions with multiplicative weight perturbations. In *Advances in Neural Information Processing Systems*, 2024.
- Tseng, A., Yu, T., and Park, Y. Training LLMs with MXFP4. In *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*, 2025.
- Ueyoshi, K., Ando, K., Hirose, K., Takamaeda-Yamazaki, S., Kadomoto, J., Miyata, T., Hamada, M., Kuroda, T., and Motomura, M. QUEST: A 7.49TOPS multi-purpose log-quantized DNN inference engine stacked on 96MB 3D SRAM using inductive-coupling technology in 40nm CMOS. In *IEEE International Solid-State Circuits Conference*, pp. 216–218, 2018.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Wang, N., Choi, J., Brand, D., Chen, C.-Y., and Gopalakrishnan, K. Training Deep Neural Networks with 8-bit Floating Point Numbers. In *Advances in Neural Information Processing Systems*, 2018.
- Wang, R., Gong, Y., Liu, X., Zhao, G., Yang, Z., Guo, B., Zha, Z., and Cheng, P. Optimizing Large Language Model Training Using FP4 Quantization. In *International Conference on Machine Learning*, 2025.
- Wen, Y., Vicol, P., Ba, J., Tran, D., and Grosse, R. Flipout: Efficient Pseudo-Independent Weight Perturbations on Mini-Batches. In *International Conference on Machine Learning*, 2018.
- Wightman, R. PyTorch Image Models. <https://github.com/rwightman/pytorch-image-models>, 2019.
- Zellner, A. Optimal Information Processing and Bayes’s Theorem. *The American Statistician*, 42(4):278–280, 1988.
- Zhang, R., Wilson, A. G., and De Sa, C. Low-Precision Stochastic Gradient Langevin Dynamics. In *International Conference on Machine Learning*, 2022.
- Zhao, J., Dai, S., Venkatesan, R., Zimmer, B., Ali, M., Liu, M.-Y., Khailany, B., Dally, W. J., and Anandkumar, A. LNS-Madam: Low-Precision Training in Logarithmic Number System Using Multiplicative Weight Update. *IEEE Transactions on Computers*, 71(12):3179–3190, 2022.
- Zhong, W., Sorscher, B., Lee, D. D., and Sompolinsky, H. A theory of learning with constrained weight-distribution. In *Advances in Neural Information Processing Systems*, 2022.

A. Setting Details for the LMD Optimizer

PyTorch Implementation LMD is easy to use because it is implemented as a PyTorch optimizer, like AdamW and Madam, and its implementation is very similar to IVON (Shen et al., 2024)¹, an optimizer for Bayesian deep learning. The simplest way to use it is shown in Listing 1.

Listing 1 The simplest implementation of LMD in PyTorch.

```

1  import torch
2  import torch.nn.functional as F
3  +from lmd.lmd import LMD
4
5  train_loader = torch.utils.data.DataLoader(train_dataset)
6  test_loader  = torch.utils.data.DataLoader(test_dataset)
7  model       = MLP()
8
9  -optimizer = torch.optim.AdamW(model.parameters())
10 +optimizer = LMD(model)
11
12 for X, y in train_loader:
13
14 + for _ in range(train_samples):
15 +     with optimizer.sampled_params():
16         optimizer.zero_grad()
17         logit = model(X)
18         loss = F.cross_entropy(logit, y)
19         loss.backward()
20
21     optimizer.step()

```

B. Experimental Details

Microscaling (MX) Data Format Settings In this study, both MXFP6 and MXFP4 formats are used as default settings. To enable training of large networks on existing GPUs, we use a custom CUDA library to emulate the MX data format (Project, 2023; Rouhani et al., 2023c). Under this emulation, low-precision matrix multiplications are performed in MXFP6 or MXFP4. In our experiments, vector operations such as activations and all backward-pass computations are carried out in bfloat16. We note that the internal state variables of the optimizer remain in FP32.

MX Data Format Emulation We applied the MX PyTorch Emulation Library² for forward-only, low-precision experiments. In this emulation (see Fig. 8), activations and weights are first converted to the specified MX data format and then dequantized to bfloat16. Matrix multiplication is performed on these dequantized values; the result is then cast to bfloat16, after which activation and normalization layers are applied. Backward propagation is carried out in bfloat16.

Madam Settings As discussed by Bernstein et al. (2020), the performance of Madam varies significantly depending on the threshold of the weight clipping. Therefore, in this paper, we clip the magnitude of Madam’s weights so that they do not exceed 1, consistent with the soft clipping applied to the weights of LMD. This is because, in Madam’s original clipping settings, the initial ℓ_2 -norm of the weights drops dramatically in the first update regardless of the gradient, making comparisons of training dynamics with LMD or AdamW impossible.

ViT Experimental Settings We employed a Vision Transformer (ViT) with an embedding dimension of 384, 6 attention heads, and 12 transformer layers. Input images were resized to 224×224 pixels and split into 16×16 patches. Instead of using the [CLS] token embedding, we applied global average pooling over patch embeddings for classification. Positional embeddings were treated as learnable parameters. We adopted the implementation from PyTorch Image Models (Wightman, 2019). All ViT experiments were run on the ImageNet dataset (Deng et al., 2009) for 200 epochs with a batch size of 4,096.

¹<https://github.com/team-approx-bayes/ivon>

²<https://github.com/microsoft/microscaling/tree/v1.1.0>

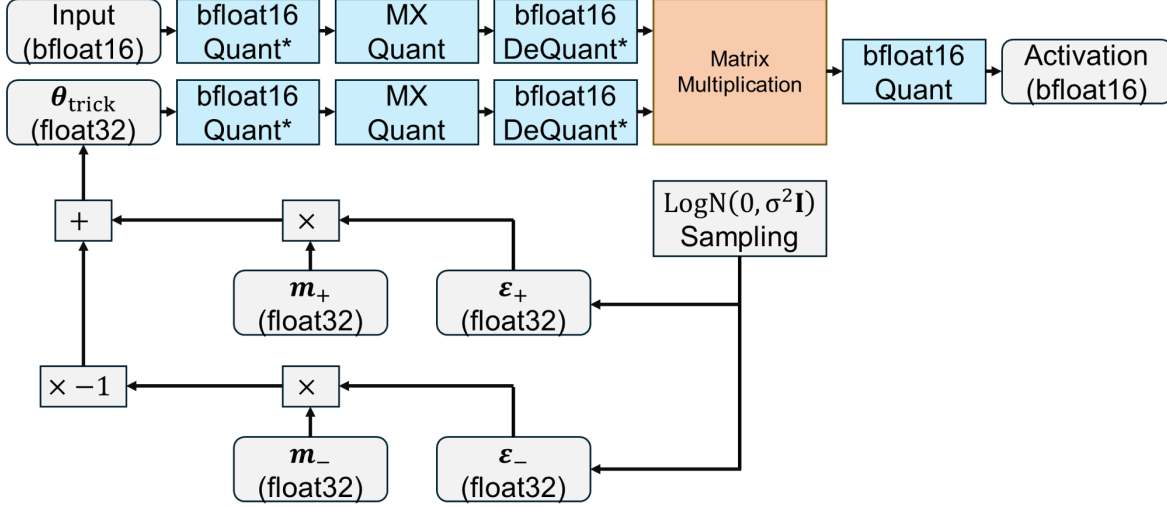


Figure 8. Overview of MX PyTorch emulation for forward passes with noise injection. Quantization (Quant) and Dequantization (DeQuant) of bfloat16 and quantization of the specified MX format are performed. The weights m_+ and m_- , as well as the sampled noise variables ε_+ and ε_- , were represented using float32. Without emulation, the steps marked with * are skipped.

We applied standard data augmentation, random resized cropping to 224×224 and random horizontal flipping. Training was performed on eight NVIDIA H100 GPUs (96 GB each), requiring about 6 to 9 hours; using the MX format emulator extended this to roughly half a day (Table 3). We distributed one GPU per node across eight nodes. In LMD experiments, each GPU samples weights from a different random seed, resulting in $J = 8$ and $S = 1$ for MC sampling. The learning rate follows a cosine-annealing schedule down to zero, with a linear warmup over the first 10,000 steps. For LMD, we set the learning rate to $\eta = 0.005$, $\sigma = 0.125$, and $m_r = 0.01 \times \exp(\frac{\sigma^2}{2})^3$. LMD did not employ gradient-norm clipping. For AdamW, we used a learning rate of 0.001, with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and a weight decay of 0.1. For Madam, the learning rate was 0.01, and parameter values were clipped to a maximum magnitude of 1. Both AdamW and Madam employed gradient-norm clipping at 1.

Table 3. ViT training time comparisons; mean $\pm$ standard deviation ($n = 3$ independent runs). This training time includes the time required to acquire data, such as calculating the weight and momentum norm.

Dataset/ Architecture	Optimizer	Forward Precision	Time (min)	Ratio
ImageNet/ Vision Transformer	AdamW	bfloat16	433.1 $\pm$ 36.8	1.0
		MXFP6	583.7 $\pm$ 2.1	1.35
	Madam	bfloat16	426.4 $\pm$ 59.1	0.98
		MXFP6	607.5 $\pm$ 3.8	1.40
	LMD (Proposed)	bfloat16	480.3 $\pm$ 69.8	1.11
		MXFP6	654.2 $\pm$ 27.6	1.51

GPT-2 Experimental Settings We trained the GPT-2 model (Radford et al., 2019) on the OpenWebText dataset (Gokaslan & Cohen, 2019) for a total of 25,000 steps, processing approximately 2.1×10^6 tokens per step (total ≈ 52 billion tokens) using 32 gradient accumulation steps. We used the nanoGPT framework (Karpathy, 2022) to train the model. Training was performed on eight NVIDIA H100 GPUs (96 GB each), requiring about 5 to 7 hours; using the MX format emulator extended this to roughly half a day and required sixteen H100 GPUs (Table 4). We assigned one GPU to each node and determined the number of nodes based on the number of GPUs to be used. In the LMD experiments, on each GPU, we sampled weights from a different random seed, and the number of MC samples per weight update was 32, matching the number of gradient accumulation steps, i.e., $J = 8$ and $S = 4$ were used. The learning rate decays to one-tenth of its initial value via cosine annealing, following a linear warmup over the first 2,000 steps. We trained all GPT-2 models without

³ $m_r = 0.01$ is expected to give the same results.

bias parameters with 124 M parameters in total. For LMD, we set the initial learning rate to $\eta = 5 \times 10^{-3}$, $\sigma = 0.125$, and $m_r = 0.01 \times \exp(\frac{\sigma^2}{2})$ (same as ViT settings). LMD employed gradient-norm clipping at 10. For AdamW, we used a learning rate of 6×10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.95$, and a weight decay of 0.1. For Madam, the learning rate was 1×10^{-2} with a weight decay of 0.1.

Table 4. GPT-2 training time comparisons; mean $\pm$ standard deviation (n = 3 independent runs). This training time includes the time required to acquire data, such as calculating the weight and momentum norm. [†] The MXFP6 experiments used 16 GPUs, not 8 GPUs, so the times and ratios cannot be simply compared.

Dataset/ Architecture	Optimizer	Forward Precision	sequence length 1024		sequence length 4096	
			Time (min)	Ratio	Time (min)	Ratio
OpenWebText/ GPT-2	AdamW	bfloat16	306.3 $\pm$ 3.6	1.0	385.3 $\pm$ 0.5	1.0
		MXFP6 [†]	620.9 $\pm$ 35.2	2.03	662.1 $\pm$ 34.7	1.72
	Madam	bfloat16	302.4 $\pm$ 0.4	0.99	383.2 $\pm$ 2.0	1.00
		MXFP6 [†]	602.9 $\pm$ 0.7	1.97	643.0 $\pm$ 0.4	1.67
	LMD (Proposed)	bfloat16	367.2 $\pm$ 7.3	1.20	443.1 $\pm$ 0.8	1.15
		MXFP6 [†]	649.2 $\pm$ 1.2	2.12	688.4 $\pm$ 0.7	1.79

C. Experiments for Small Networks

C.1. Comparison of LMD and Kiral et al. (2023) in fully connected networks

We evaluated the original Lie-Group Bayesian Learning Rule (Lie-Group BLR) algorithm of Kiral et al. (2023) using their publicly available JAX implementation⁴, while we implemented LMD and AdamW in PyTorch under the same experimental settings. We trained a fully connected network with 5 hidden layers (1024, 512, 256, 256, 256 neurons) and tanh nonlinearity. We trained on MNIST for 25 epochs with a batch size of 50. Kiral et al. (2023) did not conduct experiments with a log-normal distribution; instead, their multiplicative weight updates used a Rayleigh distribution. In their setup, the learning rate was set to 50 and the number of Monte Carlo (MC) samples to 32. To match the conditions of LMD, we reduced the number of MC samples to 1 and changed the learning rate over 50, 20, and 10. For AdamW, we used a learning rate of 1×10^{-3} with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. For LMD, we used a learning rate of 5×10^{-3} , $\beta_1 = 0.95$, $\beta_2 = 0.999$, $\sigma = 0.125$ and $m_r = 0.01 \times \exp(\frac{\sigma^2}{2})$.

Table 5 shows that, under the original proposal of Kiral et al. (2023), training is difficult even for small networks. In their experiments, they were only able to successfully train MLPs by using the large number of MC samples to stabilize learning. When the MC size is reduced to 1, as in LMD, the method either diverges or exhibits a clear degradation in performance.

Table 5. MLP performance comparisons; mean $\pm$ standard deviation (n = 3 independent runs).

Dataset/ Architecture	Optimizer	Learning Rate	MC Sample	Test Accuracy (%)
MNIST/ Multi- Layer Perceptron	Original	50	32	98.23 $\pm$ 0.03
	Lie-Group	50	1	NaN
	BLR	20	1	85.63 $\pm$ 2.99
	(Kiral et al., 2023)	10	1	62.52 $\pm$ 1.72
	AdamW	1×10^{-3}	—	98.25 $\pm$ 0.03
	LMD (Proposed)	5×10^{-3}	1	98.60$\pm$0.02

C.2. Comparison of LMD and Madam in small transformers

We compared LMD against Madam under experimental settings in which Madam can train stably. We trained a Transformer model with two hidden layers on the WikiText-2 dataset, using their publicly available implementation⁵. For Madam, we used a learning rate of 0.01. In the GPT-2 experiments, the weights were clipped to 1 to match the soft clipping used

⁴<https://github.com/team-approx-bayes/liegroups>

⁵<https://github.com/jxbz/madam>

by LMD; in this experiment, however, we set the weight clipping parameter of Madam (called scale) to 2.0 in order to prioritize the performance of Madam. For AdamW, we used a learning rate of 1×10^{-4} with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. For LMD, we used a learning rate of 5×10^{-3} , $\beta_1 = 0.95$, $\beta_2 = 0.99$, $\sigma = 0.25$ and $m_r = 0.01 \times \exp(\frac{\sigma^2}{2})$.

Table 6 shows that Madam achieves better performance than AdamW, while LMD further outperforms Madam in bfloat16 settings. Furthermore, similar to the ViT and GPT-2 experiments, LMD does not exhibit performance degradation due to low-precision forward passes (MXFP6).

Table 6. Small transformer performance comparisons; mean $\pm$ standard deviation (n = 3 independent runs). LMD uses the expected weight values during testing.

Dataset/ Architecture	Optimizer	Forward Precision	Test PPL
WikiText-2/ Transformer	AdamW	bfloat16	170.5 $\pm$ 0.3
		MXFP6	264.3 $\pm$ 81.0
	Madam	bfloat16	165.9 $\pm$ 0.2
		MXFP6	387.2 $\pm$ 3.1
	LMD (Proposed)	bfloat16	156.1$\pm$0.5
		MXFP6	155.9$\pm$0.6

C.3. Comparison of LMD and Madam in CNNs

We also trained ResNet-18 on the CIFAR-10 and CIFAR-100, using their implementation (Table 7). The experimental settings, including the ResNet-18 architecture, 300 training epochs, data augmentation, learning-rate schedule, and Madam hyperparameters, was kept the same as in the original experiments, except that the batch size was changed to 128. Madam uses a modified Batch Norm layer rather than the standard ResNet implementation. Since this modification was not supported by MX data format emulator, we also evaluated Madam with the same standard ResNet-18 used for SGD and LMD. For LMD, we used a learning rate of 0.005, $\beta_1 = 0.95$, $\beta_2 = 0.995$, and $\sigma = 0.25$.

D. Effect of the EG $\pm$ Trick

In LMD, we use the EG $\pm$ trick to preserve the expressiveness of the model under multiplicative updates. This trick also allows the weights in θ_{trick} to change sign during training. To examine the role of the EG $\pm$ trick, we evaluated LMD without this trick on GPT-2. The experimental setting is the same as in Fig. 4, except that the EG $\pm$ trick is removed by setting the m_r offset term in Eq. 16 to zero and setting gradient clipping to the same value as used for Madam.

As shown in Table 8, LMD without the EG $\pm$ trick can still train GPT-2 and achieves a lower validation loss than Madam. However, its performance is worse than that of LMD with the EG $\pm$ trick and also does not reach the performance of AdamW. This indicates that the EG $\pm$ trick is important for improving the performance of LMD.

E. Exploratory Study on Log-Magnitude Activation Normalization

The log-normal structure in LMD concerns weights, not activations. Thus, this normalization is not directly motivated by LMD itself. Nevertheless, we preliminarily tested a log-normal activation normalization for GPT-2 training under LMD. We replaced the normalization layer with $z_{\text{norm}} = \text{LayerNorm}(\log(|a| + \epsilon))$, $z = \text{sign}(a) \exp(z_{\text{norm}})$, where $\epsilon = 10^{-16}$. This normalization caused instability. A likely reason is that the log-magnitude transform introduces $\frac{\partial}{\partial a} \log(|a| + \epsilon) = \frac{\text{sign}(a)}{|a| + \epsilon}$, which can become very large near zero. Moreover, activations are basically signed and can be zero, whereas log-normal variables are strictly positive. Thus, this log-normal-based normalization is not well matched to our setting.

F. Additional Analysis of Memory Usage

F.1. Memory Usage Comparison between LMD and AdamW

LMD requires more memory than AdamW. Since peak memory depends on the implementation, we provide only a rough estimate for the tensors related to the optimizer. In LMD, the main tensors that need to be kept in memory at the same time

Table 7. CIFAR-10 and CIFAR-100 performance comparison. LMD uses sampled weight values during training and the expected value of the weight during testing.

Dataset/ Architecture	Optimizer	Forward Precision	Test Accuracy (%)
CIFAR-10/ ResNet-18	SGD	bfloat16	93.49
		MXFP6	92.70
	Madam	bfloat16	93.87
		MXFP6	15.63
	Madam (modified BN)*	bfloat16	93.75
		MXFP6	—
	LMD (Proposed)	bfloat16	94.41
		MXFP6	93.90
CIFAR-100/ ResNet-18	SGD	bfloat16	70.41
		MXFP6	69.47
	Madam	bfloat16	73.73
		MXFP6	3.73
	Madam (modified BN)*	bfloat16	73.28
		MXFP6	—
	LMD (Proposed)	bfloat16	75.39
		MXFP6	73.36

* Madam (modified BN) denotes the BatchNorm-modified network used in the original Madam setting. The MXFP6 run with modified BN is not supported by the MX data format emulator.

Table 8. Comparison of LMD with and without the $EG\pm$ trick on GPT-2. The values for AdamW, Madam, and LMD are taken from Table 2.

Method	Seq. length	Val. loss	Weight norm
AdamW	1024	2.937	392.7
Madam	4096	3.303	765.1
LMD	4096	2.926	212.9
LMD (no $EG\pm$)	4096	3.183	222.59

during training are $\mathbf{m}$, ν , θ , $\nabla \ell(\theta_{\text{trick}})$, and θ_{trick} . Their sizes are $2P$, $2P$, $2P$, $1P$, and $1P$ respectively. Therefore, the memory needed for these tensors is roughly $8P$. In contrast, AdamW keeps θ , $\mathbf{m}$, $\mathbf{v}$, and $\mathbf{g}$, which require roughly $4P$. The memory used by temporary tensors during the update step depends on the implementation. In LMD, temporary tensors, such as ε , $\mathbf{g}$, $\mathbf{r}$, and ν_{temp} , and the bias-corrected moments in AdamW can be computed layer by layer and released after use.

Thus, when activation memory is not the main cost, the memory needed for optimizer-related tensors and gradients is about twice as large for LMD as for AdamW. However, in Transformer training with long contexts or large batch sizes, activation memory often takes most of the memory (Chitsaz et al., 2024; Chen et al., 2025). In such cases, this extra optimizer memory is relatively small compared with the total peak memory.

F.2. Peak CUDA Memory Measurements on GPT-2

For GPT-2, we measured peak CUDA memory before the forward pass and during the forward, backward, and update stages, using the same setting as in Table 2 with bfloat16 (Table 9). These measurements include all GPU-resident tensors, not only optimizer states but also the GPT-2 model weights. Before the forward pass, LMD uses about $1.5\times$ more memory than AdamW. In contrast, during the forward and backward stages, where activation memory dominates, the difference between LMD and AdamW is only about 10%. In our implementation, because the number of tokens per step is matched, the 1k setting uses a batch size four times larger than the 4k setting, resulting in almost the same activation memory usage.

Table 9. Peak CUDA memory usage for GPT-2 training. Units are GB.

Stage	AdamW 1k	AdamW 4k	LMD 1k	LMD 4k
Before forward	8.06	8.09	11.78	11.88
Forward	35.02	35.05	38.73	38.84
Backward	47.30	47.33	51.02	51.12
Update	8.99	9.04	13.89	14.03

The additional memory usage of LMD mainly comes from optimizer-related tensors, and the difference from AdamW remains about 10% during the forward and backward stages. This gap may be further reduced by optimizing the LMD implementation and by using MXFP6 forward passes on real hardware.

G. Author Contributions Statement

Author list: Keigo Nishida (KN), Eren Mehmet Kırıl (EMK), Kenichi Bannai (KB), Mohammad Emtiyaz Khan (MEK), Thomas Möllenhoff (TM)

KN conceived the concept of log-normal weight dynamics, underscored its strong connection to both neuroscience and computer-science approaches to machine learning, and led the project. KN also developed the core algorithms in Section 3, conducted all experiments with some advice from TM, and drafted the manuscript with assistance from EMK, MEK, and TM. EMK and KB refined the mathematical description; Insights from MEK and TM on variational learning shaped the method’s direction; and EMK gave substantial feedback on the whole paper. The final manuscript was reviewed and approved by all authors.